

EXPERIMENTS CONFIGURATION

This is a supplementary material to our submission *JointDiffusion: Joint Representation Learning for Generative, Predictive, and Self-Explainable AI in Healthcare*. In this document we describe configuration details for the conducted experiments.

I. COMPUTATIONAL RESOURCES

Each experiment reported in this work was run on two Nvidia A100 GPUs.

II. AUTOENCODER CONFIGURATION

In our experiments, we follow the implementation of the KL-autoencoder employed by Rombach et al. [3].

a) *Model Architecture*: The base number of channels in the model is set to 128. The channel multiplier factors across different stages are [1, 2, 4, 4], which determine the progression of channel counts in the network. Each stage of the autoencoder includes two residual blocks. The encoder compresses the input sample to a $32 \times 32 \times 4$ latent representation.

b) *Model Training*: We train the autoencoder model with a base learning rate of 4.5×10^{-6} over 300,000 training steps. The batch size is set to 64. The training images have a resolution of 256×256 pixels, with RGB images ($256 \times 256 \times 3$) for the ISIC2019 dataset and grayscale images ($256 \times 256 \times 1$) for the ChestXRay14 dataset. Perceptual loss is incorporated starting from step 50,000 with a weight of 0.5, and a KL divergence loss is applied with a weight of 1.0×10^{-6} .

III. DIFFUSION MODEL CONFIGURATION

For the diffusion model, once more, we build our implementation on the work of Rombach et al. [3], adapting the UNet architecture to incorporate a joint classifier module.

a) *Model Architecture*: In our experiments, we utilize the following UNet architecture. The latent sample size is set to $32 \times 32 \times 4$. The base number of channels in the convolutional layers is 256. Each stage of the U-Net includes two residual blocks, with the channel count increasing progressively across stages by factors of 1, 2, and 4. Attention mechanisms are incorporated at resolutions of 4×4 , 2×2 , and 1×1 . For the attention mechanism, the model employs a multi-head setup with each head consisting of 32 channels.

We limit the size of the classification features extracted from the UNet model to 10,000 using an average pooling operation. The classifier component is a multilayer perceptron with two fully connected layers. The first layer uses a LeakyReLU activation function with a negative slope of 0.2.

b) *Model Training*: For experiments on the ChestXRay14 dataset with 2% of labeled samples, we use the Adam optimizer with a base learning rate of 5.0×10^{-5} . The learning rate for the classifier is set to 1.0×10^{-5} . The classification loss is scaled with a weight of 0.00005, and the classification process begins after 5000 training steps. We train the model for 200,000 steps, using a batch size of 64 for the diffusion component and 32 for the classification component. For scenarios with 5% of labeled data, we increase the classifier learning rate to 2.0×10^{-5} and the classification loss weight to 0.00015.

For higher percentages of labeled data (10%, 15%, 20%, and 100%), we use a classification loss weight of 0.001 and set the classifier learning rate to 5.0×10^{-5} .

For experiments on the ISIC2019 dataset, we use the Adam optimizer with a learning rate of 5.0×10^{-5} . The classification loss is scaled with a weight of 0.001. We train the model for 20,000 steps, using a batch size of 64 for the diffusion component and 32 for the classification component. This configuration is used for all percentages of labeled data.

IV. BASELINE METHODS CONFIGURATIONS

In this section, we provide details regarding our reproduction of baseline methods. Consistently with the experimental setups in the respective original papers, all methods employ the DenseNet121 backbone.

a) *NIH ChestXRay14*: For the experiments with the standard DenseNet121, we utilize the Adam optimizer with a learning rate of 0.03 and a batch size of 16. We apply the same augmentations as in the warm-up phase of ACPL [1], resizing the images to 256×256 pixels. The model pretrained on ImageNet is trained for 40 epochs, while the model without pretraining weights is trained for 100 epochs.

For ACPL [1], we follow the parameters specified by Liu et al.: Adam optimizer with a learning rate of 0.03, batch size of 16, 20 epochs for the warm-up training, and 10 epochs for each of the 5 ACPL loops. We use the same augmentations as Liu et al., resizing the images to 256×256 pixels. In experiments with ACPL without ImageNet pretraining weights, we use the same parameters but increase the warm-up training to 100 epochs.

For S2MTs2 [2], we follow the training configurations provided by the authors. The images are resized to 256×256 pixels. The model is pretrained on the entire training dataset in a self-supervised manner for 100 epochs, followed by an additional 10 epochs of fine-tuning as described in the original article.

b) *ISIC2019*: For the experiments with the standard DenseNet121, we use the Adam optimizer with a learning rate of 0.001 and a batch size of 32. We apply the same augmentations as described in the warm-up phase of ACPL [1], resizing the images to a resolution of 224×224 pixels. For the model pretrained on ImageNet, we train for 40 epochs, while for the model without pretraining weights, we train for 100 epochs.

For the FixMatch experiments [4], we set the learning rate to 0.0001 and the batch size to 36. The training consists of 100,000 steps and the confidence threshold is set to 0.95.

For ACPL [1], we adhere to the parameters specified by Liu et al.: a learning rate of 0.001, a batch size of 32, 40 training epochs for the warm-up training, and 20 training epochs for each of the 5 ACPL training loops. We use the same augmentations as Liu et al., resizing the images to 224×224 pixels. In experiments without ImageNet pretraining weights, we increase the number of warm-up training epochs to 100.

Similarly, for S2MTS2 [2], we once more follow the training configurations provided by the authors in their code repository.

As implemented in original paper, we resize images to a 224×224 resolution, pretrain the model on the whole training dataset in a self-supervised manner for 100 epochs, with additional 10 epochs of finetuning.

V. VISUAL COUNTERFACTUAL EXPLANATIONS - EXPERIMENTS CONFIGURATIONS

We obtain the counterfactual examples of disease indicators removal with the following procedure. First, we add noise to respective class samples running the forward diffusion process up to $t = 300$. Then we run the backward diffusion process to $t = 0$ with classifier guidance to reduce the score prediction of a given disease class with a guidance scale equal to 500.

For the experiments corresponding to disease indicators enforcing we run the forward diffusion process up to $t = 200$ and then run the denoising process to maximize the score prediction of a given disease class with classifier guidance scale equal to 200.

VI. GENERATION WITH CLASSIFIER GUIDANCE - EXPERIMENTS CONFIGURATIONS

For the experiments on generating synthetic data with increasing guidance strength we generate new samples using DDIM sampling method with number of timesteps equal to 50. For the generations with progressive guidance scale we use scale values of 0, 100, 200, 300.

REFERENCES

- [1] Liu, F., Tian, Y., Chen, Y., Liu, Y., Belagiannis, V., Carneiro, G.: Acpl: Anti-curriculum pseudo-labelling for semi-supervised medical image classification. arXiv:2111.12918 (2022)
- [2] Liu, F., Tian, Y., Cordeiro, F.R., Belagiannis, V., Reid, I., Carneiro, G.: Self-supervised mean teacher for semi-supervised chest x-ray classification (2021)
- [3] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B.: High-resolution image synthesis with latent diffusion models (2022)
- [4] Sohn, K., Berthelot, D., Li, C.L., Zhang, Z., Carlini, N., Cubuk, E.D., Kurakin, A., Zhang, H., Raffel, C.: Fixmatch: Simplifying semi-supervised learning with consistency and confidence (2020)